

RS 1: Global Split LSD Radix Sort

Source: <https://github.com/mark-poscablo/gpu-radix-sort>

Description: This implementation executes a Least Significant Digit sort by splitting the local sort, prefix sum, and global shuffle into three distinct kernels separated by global barriers. It processes 2 bits per pass and manually handles the scanning phase, making it highly dependent on global memory synchronization.

RS 2: Strided Radix Sort

Source: https://github.com/sty16/CUDA_radix_sort/blob/master/radix.cu

Description: This hybrid implementation assigns each thread a strided portion of the input array to sort independently using a local 32-pass radix sort without shared memory. Once the local sorting is complete, the algorithm performs a parallel k-way merge using a shared memory reduction tree to combine the thread-local lists into the final sorted output.:

RS 3: Warp-Synchronous Radix Sort

Source: <https://github.com/ufukomer/cuda-radix-sort/blob/master/radix-sort/kernel.cu>

Description: This implementation sorts a single warp of 32 elements by using the legacy `__ballot` intrinsic to calculate prefix sums via bitwise voting. It processes data one bit at a time using a ping-pong buffer in shared memory to reorder elements based on the voting results.

RS 4: Global Radix 2-Sort

Source: https://github.com/saikatroyc/cudaRadixSort/blob/master/kernel_radix.cu

Description: This implementation sorts the entire array one bit at a time (Radix-2) by generating a global predicate mask for each bit position across 32 iterations. It relies on a separate global prefix sum kernel to calculate write offsets for zeros and ones, followed by a scatter kernel to move data, resulting in high global memory traffic.

RS 5: Batched Key-Value Radix Sort

Source: https://github.com/Sinestro38/gpu_radix_sort

Description: This implementation performs a descending radix sort on independent batches of integer key-value pairs, where each CUDA block isolates and sorts one batch entirely using shared memory. It processes 4 bits per pass and maintains index payloads alongside keys, making it effectively a segmented sort for independent small arrays.

RS 6: Onesweep Radix Sort

Source: https://github.com/l0ngc/OneSweep/blob/main/src/gpu_onesweep.cu

Description: This implementation utilizes the "Onesweep" algorithm which fuses the histogram counting and data scattering phases into a single kernel pass per digit. It employs a decoupled look-back strategy to calculate prefix sums on the fly, eliminating the need for global barriers and significantly reducing global memory traffic compared to traditional multi-pass sorts.

RS 7: Double Scan Global Radix Sort

Source: https://github.com/xihuai18/RadixSort-Cuda/blob/master/kernel_radix.cu

Description: This implementation executes two complete scan passes for every bit iteration: one to calculate offsets for elements with a '0' bit and a second independent pass for elements with a '1' bit. This approach simplifies the scattering logic but doubles the prefix-sum workload and introduces additional latency due to host-device memory transfers required to link the two scan phases.

RS 8: Transposed Radix 8 bit sort

Source: <https://github.com/gcuong98/GPURadixSort/blob/master/sort.cu>

Description: This implementation processes 8 bits per pass (256 bins) to reduce the total number of global passes to just four. To handle the massive volume of histogram data, it employs a Matrix Transpose kernel to reorganize block-level counters in global memory, enabling efficient parallel prefix sums across all thread blocks before the final scatter phase.

RS 9: Signed Integer Radix Sort

Source: <https://github.com/richursa/CudaRadixSort/blob/master/radixSortCuda.cu>

Description: This implementation adds support for signed integers by partitioning the dataset into distinct positive and negative subarrays using the sign bit. It sorts each partition independently using a Radix-2 approach powered by a global Hillis-Steele scan (step-efficient prefix sum) before concatenating the results.

RS 10: Host Chained Radix Sort

Source: <https://github.com/metzzo/RadixSortCUDA/blob/master/sort.cu>

Description: This implementation executes a Radix-2 sort by serializing the scanning phase into a sequence of single-block kernel launches. The host orchestrates the dependency chain by launching a separate scan kernel for each data chunk, where each kernel reads the running total from global memory, adds its local reduction, and writes the new total for the subsequent kernel.

RS 11: Single Block Blelloch Sort

Source: <https://github.com/darwich6/RadixSort/blob/main/radixSort.cu>

Description: An educational "toy" implementation restricted to a single CUDA thread block, designed for sorting very small arrays. It manually implements a work-efficient Blelloch scan (Upsweep/Downsweep) for the prefix sum phase and relies on heavy host-device synchronization loops to coordinate the predicate, scan, and scatter kernels.

RS 12: CUB Segmented Block Radix Sort Source: (Generated by Gemini 3) Description: This implementation sorts independent data chunks in isolation using CUB block primitives and register tiling. It processes multiple items per thread to maximize register usage and shared memory bandwidth without global synchronization.

RS 13: Shared Memory Atomic Radix Sort Source: (Generated by Gemini 3) Description: This variant sorts data locally within shared memory by using atomic add operations on counters instead of a prefix scan. It calculates destination indices dynamically, placing heavy stress on the hardware atomic units rather than arithmetic units.

RS 14: Dynamic Parallelism Radix Sort Source: (Generated by Gemini 3) Description: This implementation uses CUDA Dynamic Parallelism to launch child kernels recursively from the GPU for each radix bin. It manages the entire sorting lifecycle on the device, stressing the runtime environment and kernel launch latency hiding mechanisms.

RS 15: Register-Only Warp Shuffle Sort Source: (Generated by Gemini 3) Description: This implementation sorts small arrays entirely within the GPU register file using warp shuffle intrinsics to avoid memory access. It achieves high throughput for tiny datasets by relying exclusively on register-to-register communication.

RS 16: Device-Side Recursive MSD Sort Source: (Generated by Gemini 3) Description: This Most Significant Digit sort employs device-side recursion where threads partition data and recursively call the sort function. It heavily stresses the GPU function call stack and instruction cache due to divergent control flow.

RS 17: Texture Cache Radix Sort Source: (Generated by Gemini 3) Description: This variant forces all data reads through the GPU read-only texture cache using specific intrinsics. It is designed to test the performance and resilience of the non-coherent texture memory path during sorting operations.

RS 18: Hybrid Radix-Bitonic Sort Source: (Generated by Gemini 3) Description: This hybrid approach sorts local blocks using radix sort in shared memory and then merges them using a global bitonic sort network. It combines the linear speed of radix sort with the stability of comparator-based sorting for the global phase.

RS 19: Constant Memory Lookup Sort Source: (Generated by Gemini 3) Description: This implementation stores histogram offsets and radix tables in the constant memory cache rather than global memory. It relies on the constant cache broadcasting mechanism which is optimized for uniform access but sensitive to thread divergence.

RS 20: Strided Memory Access Radix Sort Source: (Generated by Gemini 3) Description: This implementation intentionally sorts data using a non-coalesced strided memory access pattern. It is designed to maximize pressure on the memory controllers and translation lookaside buffers to test worst-case memory system behavior.

RS 21: Hybrid Implementation:

Source: https://github.com/yulyulyharuka/cuda/blob/master/src/my_radix.cu

Description: This hybrid implementation splits the sorting workload by offloading only the predicate generation and index calculation to the GPU, while performing the prefix scan and final data permutation on the host CPU. It requires transferring the entire dataset and auxiliary arrays across the PCIe bus for every single bit iteration, creating a distinct heavy-I/O bottleneck profile.